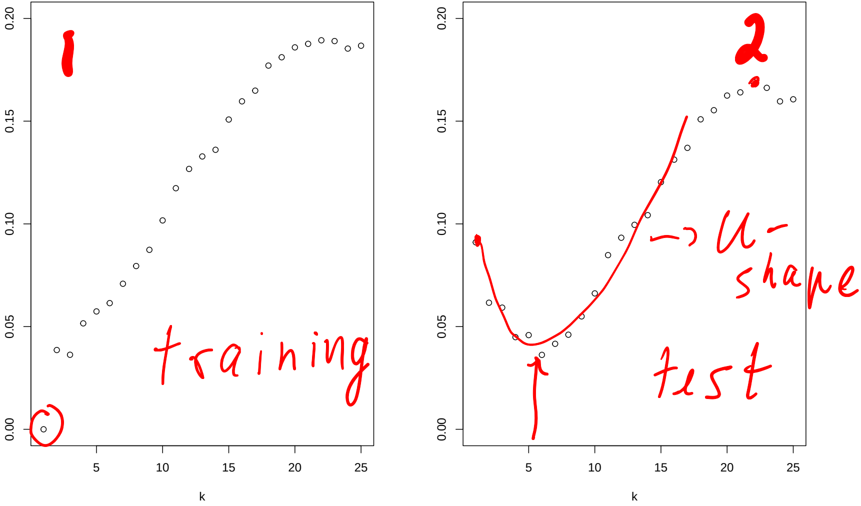
STA314, Lecture 3

September 28, 2021
Stanislav Volgushev

Test and training error: R Example

R code: `Lecture3-TestTrainingErrorExample.R`

Test and training error on simulated data: which is which?



Plot shows test and training error for k-nn on a simulated data for different values of k. Which is which?

Recap: L-fold cross-validation

Let $\mathcal{K} = \{k_1, ..., k_C\}$ denote set of candidate values for k in k-nn, i.e. we want to select the best one (in terms of test error) of those values.

---

1. Divide data into L parts (folds) of roughly equal size. Call folds $S_1, ..., S_L$.
2. For $\ell = 1, ..., L$, $k \in \mathcal{K}$, compute the k-nn estimator based on data $\{(x_i, Y_i) : i \notin S_\ell\}$ (data not in fold S_ℓ). Call resulting estimator $\hat{f}_{\ell,k}$, $k \in \mathcal{K}$.

   fold k from k-nn

3. Compute test error on ℓ'th fold

   no data from S_ℓ

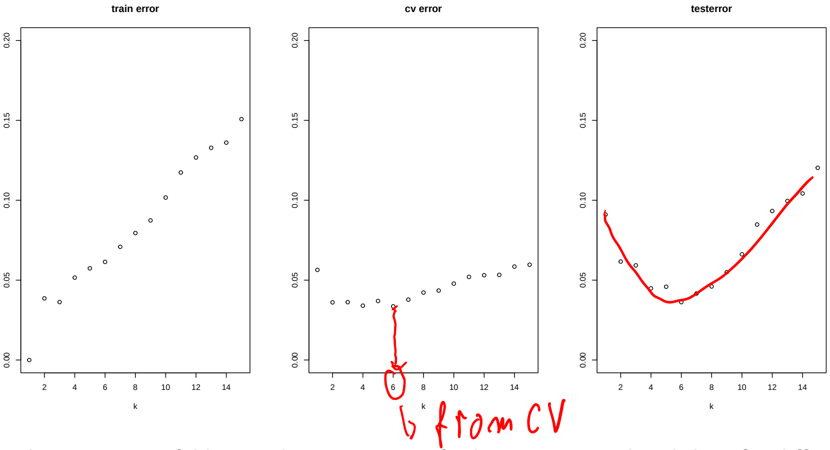
$$e_\ell(k) = \sum_{i \in S_\ell} (\hat{f}_{\ell,k}(x_i) - Y_i)^2.$$

   from S_ℓ

4. The selected k is the value with smallest resulting error $\sum_{\ell=1}^{L} e_\ell(k)$ over $k \in \mathcal{K}$, or

$$k = \arg\min_{k \in \mathcal{K}} \underbrace{\sum_{\ell=1}^{L}}_{\text{sum over folds}} e_\ell(k).$$

Test, training, and cv error on simulated data



Plot shows test, 10-fold cv and training error for k-nn on a simulated data for different values of k.

Recap: the one standard error rule with L-fold cross-validation

1. For each $k \in$ and each $\ell = 1, \ldots, L$ compute $e_\ell(k)$ as in L-fold cross-validation.
2. Find k_0 which minimizes $L^{-1} \sum_{j=1}^{L} e_j(k)$.
3. Compute $\widehat{sd}(k_0)$, the sample standard deviation of $e_1(k_0), \ldots, e_L(k_0)$.
4. Select the largest k^* among those k which satisfy

"sample mean"

$$L^{-1} \sum_{j=1}^{L} e_j(k) \leq L^{-1} \sum_{j=1}^{L} e_j(k_0) + L^{-1/2} \widehat{sd}(k_0)$$

"error in sample mean"

Question: will this select larger or smaller k compared to CV with no 1 SE?

yes

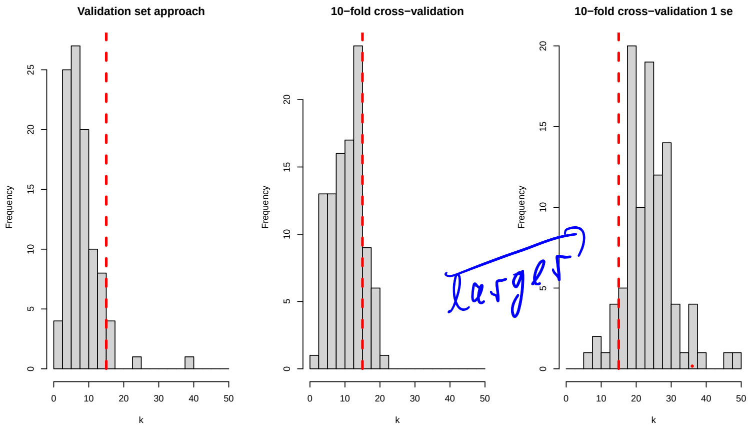
always true for k_0:

$$L^{-1} \sum e_j(k_0) \leq L^{-1} \sum e_j(k_0) + L^{-1/2} \widehat{sd}(k_0)$$

$$\iff 0 \leq L^{-1/2} \widehat{sd}(k_0) \checkmark$$

$$sd\left(\frac{1}{L} \sum_{\ell=1}^{L} e_\ell(k)\right) = \frac{1}{L^{1/2}} sd(e_1(k))$$

Validation set approach, 10-cv, 1-se 10-cv



larger

Comparing selected values for k in simulated data based on the validation set approach, 10-fold cross-validation and 10-fold cross-validation with one standard error rule. Setup: simulate 100 independent data sets. For each data set select k based on different criteria. Dotted red line: optimal k from earlier simulations (see R code).

Model assessment

- k-nn: a way to learn regression function $\hat{f}$ from data. Depends on tuning parameter k.

- Way to select tuning parameter: different variants of cross-validation. Applicable to k nearest neighbours and to selecting tuning parameters in any other methods.

- So far we have evaluated performance of methods based on simulation or our knowledge of the true regression function.

Next questions:

- How do we compare the performance of methods on real data where it is impossible to simulate new data and we don't know the true regression function?
- How will our selected method perform on 'new' data that were *never* used in the whole training process (including model or tuning parameter selection)?

The second question is related to *model assessment*. It can be important, for instance when we report the performance to collaborators/employers.

Protocol for evaluating methods on a real data set

Step 1: randomly divide data set into a training and test set. Set test set aside.

Step 2: use the training set to fit candidate models. This includes selecting tuning parameters such as K in K-nn regression via cross-validation. *Do not use any data from the test set for this step.*

Step 3: Use the final models obtained in Step 2 to predict data in the test set and compute the test error. Use this error to compare models without further tuning.

data

test

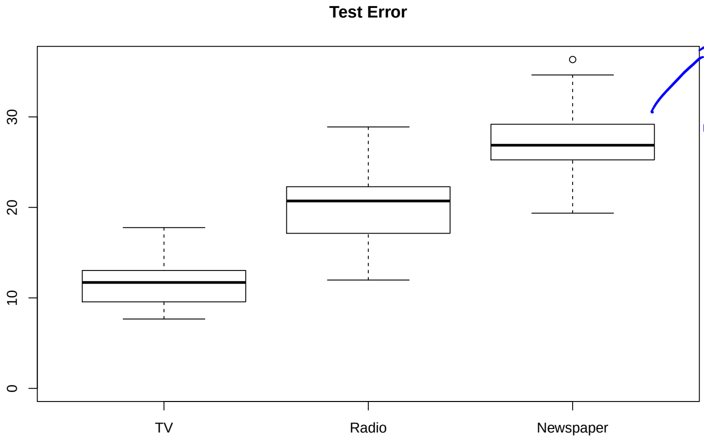
training

train model (incl. tuning param.)

- Setting aside around 20% − 30% of the data for the test set is typical.
- **The test set should never be used to tune parameters.** It appears **only** at the final stage when models are compared.
- Since the split into test and training set is random, the above procedure will produce results that depend on the split. To get a better idea of the overall performance of our model we will use repeated splitting in lectures. However, this is not standard and often not feasible.

Example: evaluating the performance of different models for Sales

Setting: run k-nn with k selected by cross-validation using TV, Radio or Newspaper to predict Sales. Split data into 20% test and 80% training once with given seed (see R code). Gives estimated errors: 14.3 (TV), 24.8 (Radio), 36.3 (News), 39.0 (mean).



box plots

Plot shows errors resulting from 25 different (random) splits. Based on this plot, which model would you use? → TV, has smallest Test err.

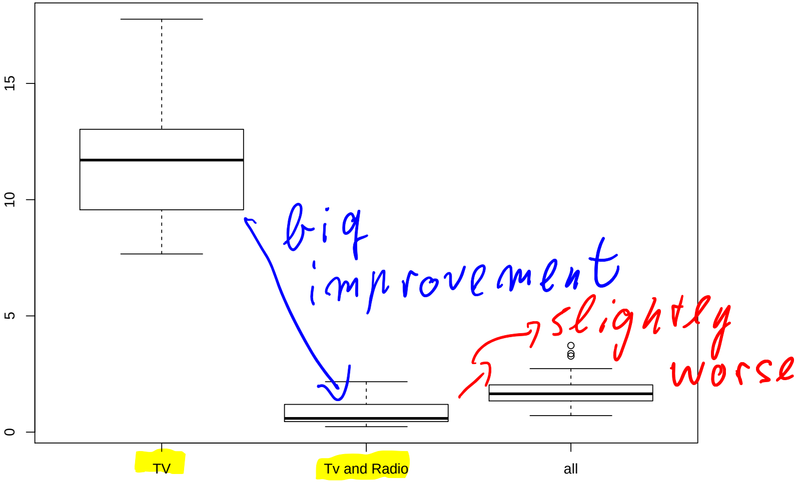
K-nn regression continued

An idea to try to improve results on the previous slide: use several predictors simultaneously!

$a = (a_1, \ldots, a_d)$

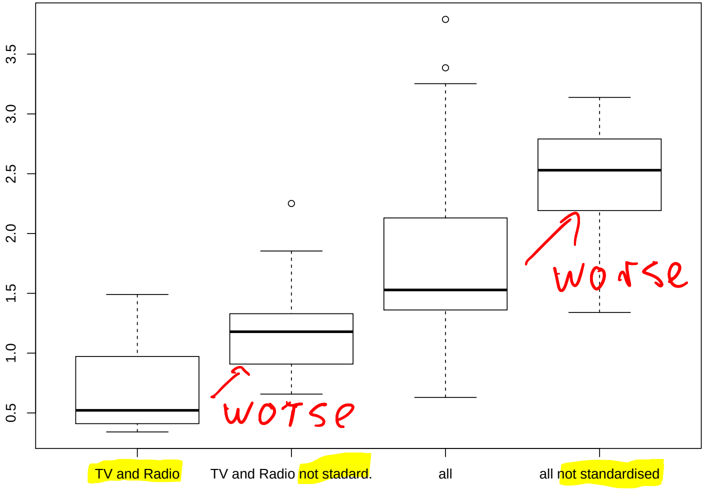
- Basic idea of k-nn: pick the *k* closest points for prediction. Using any distance in $\mathbb{R}^d$ this idea can be applied to multivariate predictors. Typical: use Euklidean distance. $\rightarrow |a - b| = \left(\sum_i (a_i - b_i)^2\right)^{1/2} \quad a, b \in \mathbb{R}^d$
- Some care must be taken with scale of individual predictors. Example: a maximal amount of 300 units was spent on TV advertisement, but only a maximal amount of 50 units on Radio advertisement.
- Unless there are good reasons to not do that, all predictors should first be standardised to have the same scale.

Advertising: K-nn with several predictors



k selected by 10-fold cross-validation. Including Radio and TV in the same model leads to substantial improvement. Additionally including Newspaper makes things worse.

Advertising: K-nn with several predictors, scaling vs no scaling



Same as before, comparison of using scaled and not scaled predictors. Scaling predictors leads to better results in this example.

$$\begin{pmatrix} TV_1 \\ News_1 \end{pmatrix} \cdots \begin{pmatrix} TV_n \\ News_n \end{pmatrix}$$

$$\rightarrow \quad \widetilde{TV}_i = \frac{TV_i - \overline{TV}}{sd(TV)}$$

Potential drawbacks of k-nn

- Curse of dimensionality: does not work when there are many predictors, especially of some of those predictors are not important.
- Difficult to work with qualitative predictors (student status, gender, color of a car etc).
- Resulting model not very interpretable.

The next set of models will help to avoid some of the drawbacks above (but might have other drawbacks instead).

Linear models

'All models are wrong but some are useful' - George Box

Linear regression models - introduction/recap

Recall: we have assumed regression model

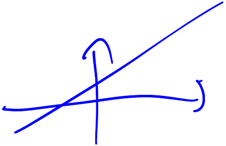
$$y_i = f(x_i) + \varepsilon_i.$$

K-nn makes smoothness assumptions on f and takes local averages

Linear regression assumes a 'linear' form for f.

Simple linear regression for one-dimensional x_i assumes

$$f(x_i) = b_1 + b_2 x_i$$



where b_1, b_2 unknown parameters that need to be learned from data.

Multiple linear regression: if $x_i = (1, x_{i,1}, ..., x_{i,d})^\top$ are (d+1)-dimensional vectors assume

intercept

$$f(x_i) = x_i^\top b = b_1 + b_2 x_{i,1} + ... + b_{d+1} x_{i,d}$$

with parameter vector $b = (b_1, ..., b_{d+1})^\top$. → unknown

- notation $x_i = (1, x_{i,1}, ..., x_{i,d})^\top$ including 1 as the first entry is for notational convenience. This is not uniform across different sources, be careful!

Ex.: $(1, TV_i)$

Estimating multiple linear regression

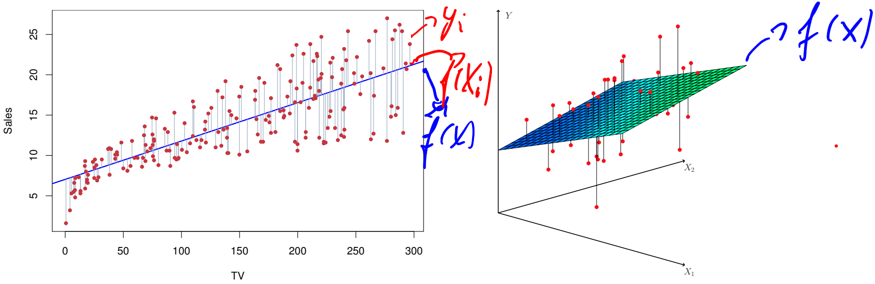
$$f(x_i) = x_i^\top b = b_1 + b_2 x_{i,1} + \ldots + b_{d+1} x_{i,d}$$

Estimation: minimize *residual sum of squares* (how does this relate to training error?)

$$RSS(b) := \sum_{i=1}^{n} \left(y_i - \underbrace{x_i^\top b}_{f(x_i)} \right)^2, \quad = \text{training error for function } f(x_i) = x_i^\top b$$

i.e. define

$$\hat{b} = \text{argmin}_{b \in \mathbb{R}^{d+1}} RSS(b).$$



Left: linear regression with $d = 1$ blue line: $\hat{f}$, right: $d = 2$, plane: $\hat{f}$.

Multiple linear regression: some formulas

Let $x_i = (1, x_{i,1}, ..., x_{i,d})^\top$. Define

$$\mathrm{X} := \begin{pmatrix} 1 & x_{1,1} & x_{1,2} & \dots & x_{1,d} \\ \vdots & \vdots & \dots & \vdots & \\ 1 & x_{n,1} & x_{n,2} & \dots & x_{n,d} \end{pmatrix} \in \mathbb{R}^{n\times(d+1)}, \quad Y := \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}.$$

$a \in \mathbb{R}^d$

$\|a\|_2 = \left(\sum_{i=1}^{d} a_i^2\right)^{1/2}$

RSS has representation (see STA302)

$$RSS(b) = \sum_{i=1}^{n} \left(y_i - x_i^\top b\right)^2 = \|Y - \mathrm{X}b\|_2^2.$$

$RSS(b)$ has unique minimizer if and only if *Gram matrix* $\mathrm{X}^\top \mathrm{X} \in \mathbb{R}^{(d+1)\times(d+1)}$ is invertible. In that case solution is given by (see STA302)

$$\hat{b} = \mathrm{argmin}_{b\in\mathbb{R}^{d+1}} \|Y - \mathrm{X}b\|_2^2 = (\mathrm{X}^\top \mathrm{X})^{-1}\mathrm{X}^\top Y.$$

$X^\top X \hat{b} = X^\top Y$

no unique sol.

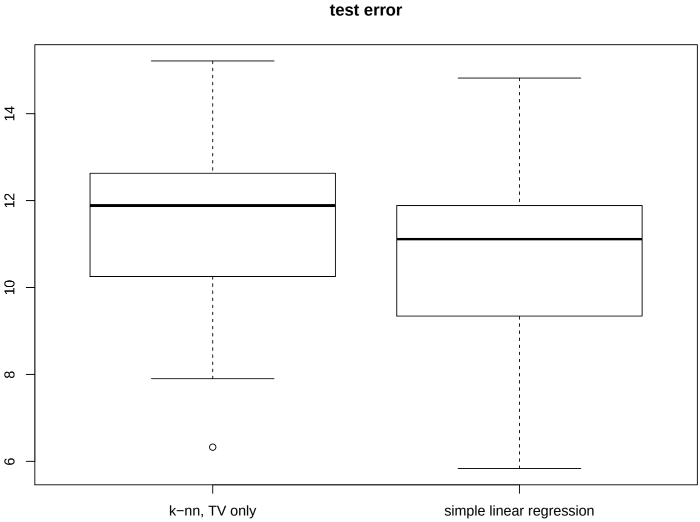
- Explicit solution, no numerical optimization required!
- Easy to compute even for large data sets.
- Question: what can you say about $\hat{b}$ if the Gram matrix is not invertible?

not unique

Multiple linear regression in R

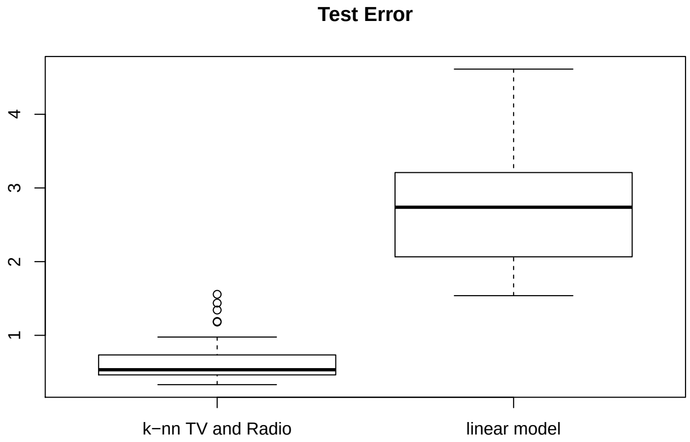
See R code, file `STA314-2020-Lecture03-rExamples-lm`

Comparison with k-nn: one predictor



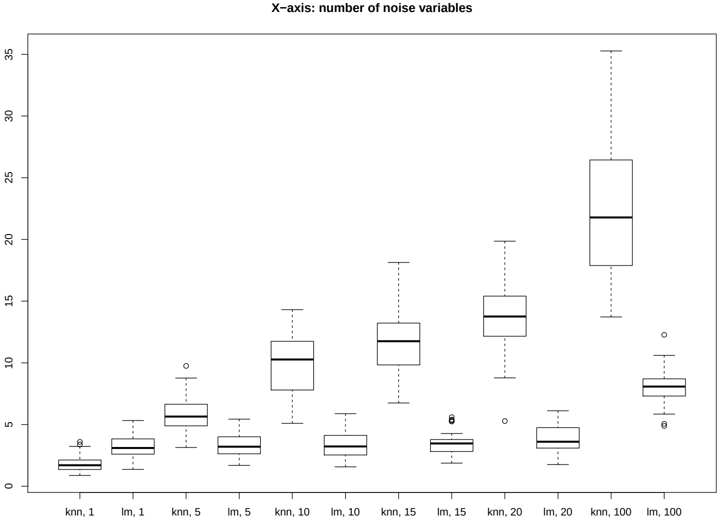
Comparison between linear model with just TV and k-nn with just TV (k selected via 10-fold cross-validation).

Comparison with k-nn: TV, Radio and Newspaper included



Comparison between linear model and k-nn with 2 predictors, TV and Radio.

Comparison with k-nn: adding many noise variables



Comparison between linear model and k-nn with different number of noise variables.